VIÑA TACAMA S.A. · EL TAMBO DE TACAMA · ICA, PERÚ

TacamaMesa

Sistema de atención digital del restaurante: carta por QR, maridaje, cocina en tiempo real e inventario de la cava.

Informe del sistema y de su despliegue en producción

Proyecto académico · Universidad Tecnológica del Perú (UTP) · Arquitectura Empresarial (TOGAF ADM)
Documento generado el 12 de noviembre de 2026 · Sistema desplegado y validado en producción

Contenido

1. **Resumen ejecutivo**
2. **Contexto y objetivos**
3. **Qué hace el sistema** (comensal, personal por rol, reglas de negocio)
4. **Arquitectura** (microservicios, datos, mensajería, tiempo real)
5. **Tecnologías utilizadas**
6. **Seguridad y privacidad**
7. **Despliegue en producción** (cómo quedó armado)
8. **¿Por qué un VPS de Contabo y no Supabase u otras opciones gratuitas?**
9. **¿Por qué Vercel y no otras plataformas?**
10. **Validación realizada** (pruebas de extremo a extremo)
11. **Riesgos, limitaciones y recomendaciones**
12. **Anexo: operación diaria y estructura del repositorio**

1. Resumen ejecutivo

TacamaMesa digitaliza la experiencia del comensal en **El Tambo de Tacama**, el restaurante de la viña más antigua de Sudamérica (fundada en 1540, Ica). El cliente escanea un código QR en la mesa, declara sus alergias, explora una carta multiidioma de 104 ítems con maridajes de vinos y piscos, y envía su pedido directamente a cocina. Cocina, mozos, sommelier, almacén y administración trabajan desde un portal del personal con permisos por rol.

6	2	104	112	5
microservicios NestJS	aplicaciones web (React)	platos y bebidas	reglas de maridaje	idiomas (es, en, fr, pt, de)

Estado actual: el sistema está desplegado y funcionando en producción. Los dos frontends están en **Vercel** y todo el backend (6 servicios, PostgreSQL, Redis, RabbitMQ) corre en un **VPS de Contabo** con Docker Compose y HTTPS automático. Se verificó de extremo a extremo: pedido del comensal, llegada a cocina por WebSocket en unos 300 ms, avance de estados, cierre de mesa, y conservación del seguimiento al recargar la página.

> **Idea central de las decisiones de despliegue.** El backend no es una aplicación simple sino una *arquitectura de microservicios con procesos permanentes* (6 servicios Node, WebSockets, cola de mensajes, caché y una base con 6 esquemas). Eso exige un servidor donde correr contenedores continuamente; por eso el backend va en un VPS. En cambio, los frontends son archivos estáticos, el caso ideal para una red de distribución como Vercel. Las secciones 8 y 9 explican el razonamiento y las alternativas.

2. Contexto y objetivos

Trabajo universitario desarrollado bajo el marco **TOGAF ADM** (visión, arquitectura de negocio, de datos, de aplicaciones y de tecnología). Cinco objetivos estratégicos:

1. **Digitalizar la atención** con QR en mesa, sin apps nativas (PWA en el navegador).
2. **Potenciar la venta cruzada** con el maridaje enológico de cada plato.
3. **Garantizar la seguridad alimentaria**: alergias obligatorias y priorizadas en cocina en tiempo real.
4. **Sincronizar el inventario** de la cava con eventos asíncronos y una bodega simulada (sandbox).
5. **Asegurar trazabilidad** con auditoría inmutable retenida por 12 meses.

3. Qué hace el sistema

3.1 Experiencia del comensal (aplicación web progresiva)

- **Acceso por QR de mesa** (por ejemplo `?mesa=QR-MESA-003`), sin cuenta ni descarga. Hay 6 mesas en tres zonas: Terraza Principal, Jardín de Viñedos y Salón Bodega 1540.
- **Aviso de privacidad previo** con dos autorizaciones separadas (condiciones generales y datos de salud), conforme a la Ley N.º 29733.
- **Perfil por comensal**: peruano o extranjero; alergias obligatorias; gustos opcionales; mesas de 1 a 8 personas con perfil individual y pedido asignable por persona.
- **Carta de 104 ítems** en 5 idiomas (según el dispositivo), con fotos, segmentada por desayuno y almuerzo, con información nutricional referencial y alérgenos por plato.
- **Recomendaciones** de entrada, fondo, postre y bebida, y **maridaje** de vinos y piscos de la casa con aviso de stock ("¡Últimas 2 botellas!").
- **Personalizar platos** (quitar ingredientes, marcar alergia, pedido especial) y **cambiar o anular** durante los 5 minutos posteriores a confirmar.
- **Seguimiento en vivo** del pedido (recibida, en cocina, lista, servida), **pedir la cuenta**, calificar (CSAT) y elegir método de pago.

3.2 Portal del personal, por rol

Rol	Qué puede hacer
Administrador	Todos los módulos: KPIs y auditoría, cocina, mesas, carta y precios, sommelier, inventario y cuentas (alta, cambio de rol, restablecer contraseña, desactivar).
Jefe de salón	Mesas, cocina, carta y precios, KPIs, inventario. Puede reasignar mesas.
Mozo	Mesas: "Atender esta mesa", confirmar entrega, cobrar, liberar mesa (con o sin cuenta).
Cocina	Pantalla KDS en tiempo real: comandas, alertas de alérgenos, iniciar preparación y marcar lista.
Sommelier / Almacén	Inventario de etiquetas (stock, umbrales, agotados) y reglas de maridaje.

3.3 Reglas de negocio implementadas

Regla	Descripción
RN01	Declaración de alergias obligatoria; cocina ve la alerta antes de preparar.
RN03	Las sugerencias de maridaje solo incluyen etiquetas con stock mayor que cero.
RN05	El maridaje presenta al menos una opción accesible y una premium.
RN06 / RN07	Al confirmar una comanda se descuenta stock por eventos; si se anula, se repone.
RN11	Los registros de auditoría son inmutables y se retienen 12 meses.

4. Arquitectura

Arquitectura de **microservicios** orientada a eventos, en un monorepo (Turborepo + pnpm). Cada servicio es dueño de sus datos (patrón *base de datos por servicio*, aquí un esquema de PostgreSQL por servicio).

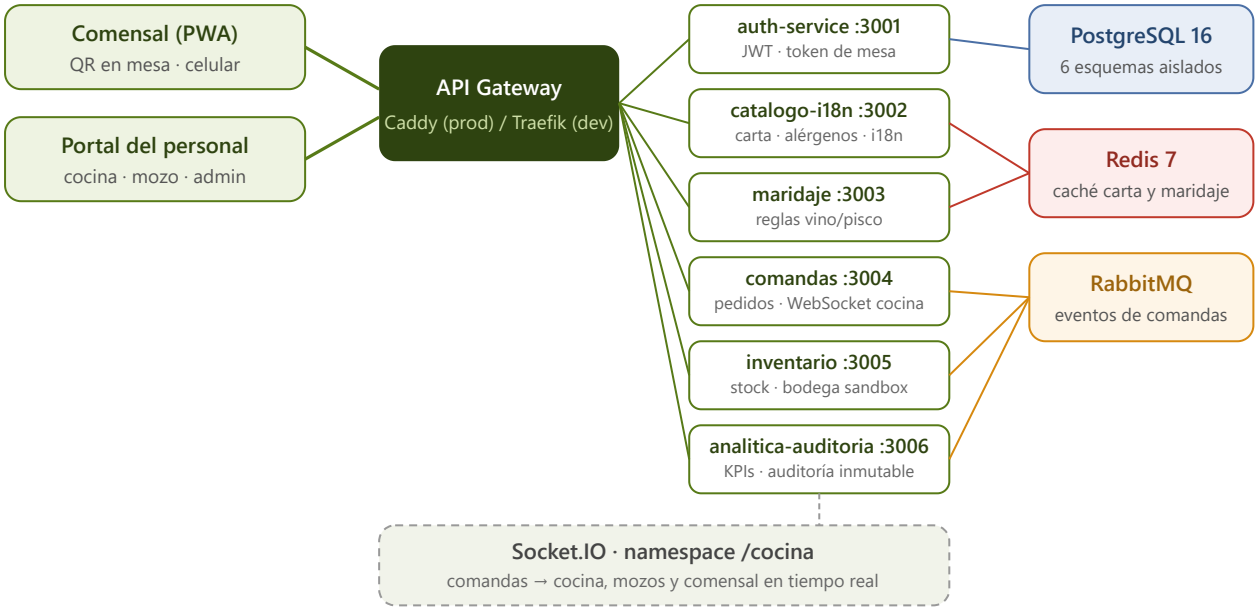


Figura 1. Arquitectura lógica de TacamaMesa.

4.1 Microservicios

Servicio	Puerto	Responsabilidad
`auth-service`	3001	Autenticación JWT del personal, cuentas, token de mesa por QR, sesiones de mesa.
`catalogo-i18n-service`	3002	Carta, platos, alérgenos, traducciones, fotos, disponibilidad y precios. Caché en Redis.
`maridaje-service`	3003	Reglas y recomendaciones de vino/pisco por plato; etiquetas. Caché en Redis.
`comandas-service`	3004	Pedidos, validación de alérgenos, estados, cuenta y pago, valoraciones; gateway WebSocket de cocina.
`inventario-service`	3005	Stock de etiquetas, movimientos, umbrales; consume eventos de comandas; sandbox de bodega.
`analitica-auditoria-service`	3006	KPIs (venta cruzada, CSAT) y bitácora de auditoría inmutable.

4.2 Datos

Una sola instancia de PostgreSQL con **seis esquemas** (`auth`, `catalogo`, `maridaje`, `comandas`, `inventario`, `analitica`), cada uno accesible solo por su servicio mediante Prisma ORM. Esto da el aislamiento lógico de "base por servicio" con la simplicidad operativa de un único servidor de base de datos.

4.3 Eventos y tiempo real

- **RabbitMQ** (exchange `tacama.events`): cuando se confirma una comanda, `comandas-service` publica el evento; `inventario-service` descuenta stock y `analitica-auditoria-service` registra el hecho. Las anulaciones reponen el stock.
- **Socket.IO** (namespace `/cocina`): la cocina recibe `cocina:nueva_comanda`; los comensales reciben `cocina:estado_cambiado`. El comensal además consulta cada 6 s como respaldo si el socket falla.

5. Tecnologías utilizadas

Capa	Tecnología
Frontend	React 18, Vite 5, TypeScript, TailwindCSS, Lucide Icons, Socket.IO Client
Backend	NestJS 10, TypeScript 5, Prisma ORM, Swagger/OpenAPI, class-validator
Datos y caché	PostgreSQL 16 (esquemas por servicio), Redis 7
Mensajería y tiempo real	RabbitMQ 3 (AMQP), Socket.IO (WebSocket)
Gateway	Traefik v3 en desarrollo local; Caddy 2 con HTTPS automático en producción
Integración externa simulada	WireMock (sandbox de la bodega de Viña Tacama)
Orquestación y repositorio	Docker Compose, Turborepo, pnpm workspaces, GitHub Actions (CI)
Pruebas	Jest (unitarias), Supertest (e2e), ESLint

6. Seguridad y privacidad

6.1 Controles existentes

- **Autenticación y roles:** JWT para el personal; cada rol ve solo sus módulos y los endpoints sensibles (cobrar, liberar mesa, administrar cuentas) validan el rol en el servidor. Contraseñas con **bcrypt**; el sistema nunca las muestra.
- **Token de mesa:** el comensal opera con un token temporal de 4 horas ligado a su mesa, sin cuenta personal.
- **Transporte:** en producción todo el tráfico va por **HTTPS** (certificado de Let's Encrypt gestionado por Caddy).
- **CORS restringido** a los dominios de Vercel; validación de entradas con class-validator; filtro global de errores.
- **Red interna:** PostgreSQL, Redis y RabbitMQ *no* publican puertos hacia Internet; solo Caddy expone 80 y 443. Firewall (UFW) con 22, 80 y 443.
- **Auditoría inmutable** y registro del motivo al liberar una mesa sin cuenta.

6.2 Protección de datos personales (Ley N.º 29733)

- No se piden DNI, teléfono, correo ni dirección al comensal; no hay cuentas de comensal ni cookies publicitarias.
- Perfil y gustos viven solo en el navegador (`sessionStorage`) y se borran al cerrar la pestaña. El pedido activo también (desde el arreglo de recarga).
- Las alergias son dato de salud: se piden con **autorización separada**, y cada comanda guarda el consentimiento y la **versión de los términos** aceptados.
- El proyecto documenta una lista de pendientes legales (responsable, contacto ARCO, registro del banco de datos).
  El texto de la política y sus plazos deben revisarse con un abogado antes de operar con clientes reales.

> **Hallazgos de seguridad que requieren acción:**
>
> - **Cuentas de ejemplo con contraseñas públicas.** El repositorio es público y documenta las contraseñas de las cuentas que crea el seed (administrador, jefe, mozos, cocina, sommelier, almacén). Con el sistema en Internet, cualquiera podría entrar como administrador. **Deben cambiarse desde el módulo Cuentas antes de usar el sistema con datos reales.**
> - **Contraseña de root del VPS.** Se compartió durante la configuración; debe reemplazarse (comando `passwd`) y, a futuro, usarse claves SSH.
> - **Swagger** (`/api/docs`) conviene desactivarlo en producción si no se necesita.

7. Despliegue en producción

El despliegue combina dos plataformas: **Vercel** para los frontends y un **VPS de Contabo** para el backend.

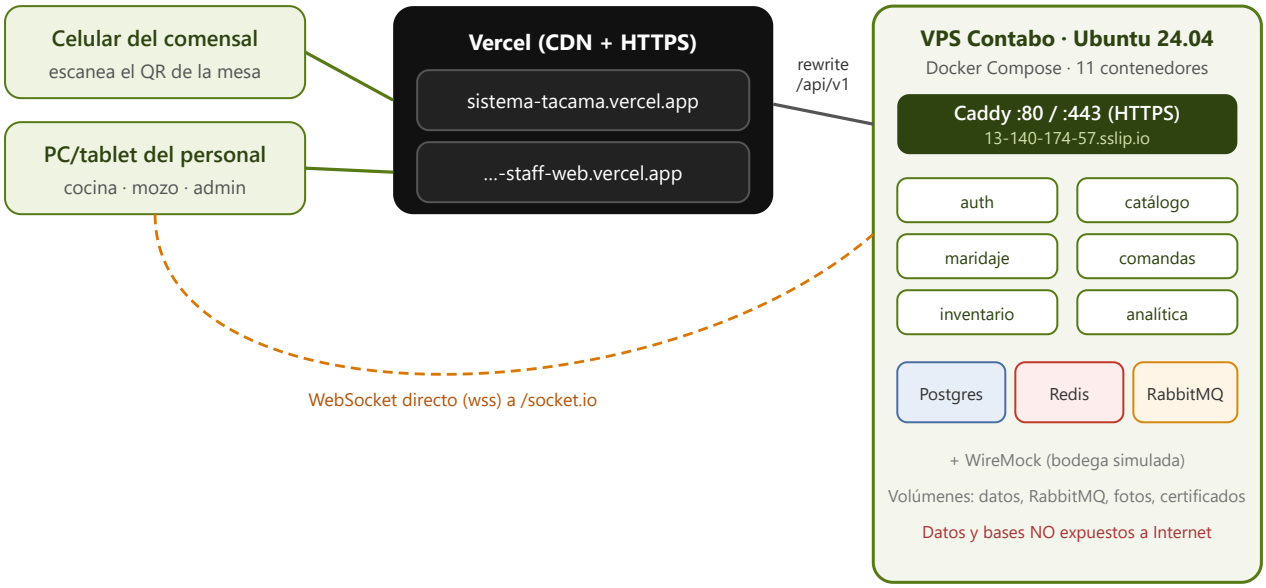


Figura 2. Despliegue: frontends en Vercel, backend en el VPS. Las llamadas REST pasan por los *rewrites* de Vercel; el WebSocket se conecta directo al VPS.

7.1 Cómo se conectan las dos partes

- **REST:** los frontends llaman a rutas relativas `/api/v1/...`. Un archivo `vercel.json` por aplicación reenvía (*rewrite*) esas rutas hacia el dominio HTTPS del VPS, de modo que el navegador solo ve su propio dominio.
- **WebSocket:** los rewrites de Vercel no transportan WebSockets, así que el cliente se conecta directo al VPS mediante la variable `VITE_SOCKET_URL` (se inyecta en tiempo de compilación). Por eso el VPS necesita HTTPS válido.
- **HTTPS sin dominio propio:** se usa `13-140-174-57.sslip.io`, que resuelve a la IP del VPS; Caddy obtiene el certificado de Let's Encrypt automáticamente.
- **CORS:** la variable `CORS_ORIGINS` de los servicios lista los dos dominios de Vercel.

7.2 Flujo de despliegue

Parte	Cómo se actualiza
Frontends	`git push` al repositorio → el fork conectado a Vercel debe sincronizarse → Vercel compila y publica automáticamente cada app.
Backend	En el VPS: `git pull` y `docker compose -f docker-compose.prod.yml up -d --build`. Cada servicio ejecuta `prisma db push` al iniciar para crear o actualizar sus tablas sin borrar datos.
Datos iniciales	Se cargan una sola vez con el seed de cada servicio (carta, etiquetas, mesas y cuentas de ejemplo).

> **Una sola imagen para 6 servicios.** Un único `Dockerfile.prod` parametrizado por servicio compila los paquetes compartidos, genera el cliente Prisma y construye cada microservicio. Es lo que hace reproducible el despliegue.

8. ¿Por qué un VPS de Contabo y no Supabase u otras opciones gratuitas?

> **Nota de transparencia.** La elección concreta de Contabo la hizo el equipo: el servidor ya estaba contratado. Lo que sigue explica las **razones técnicas** por las que un VPS es la opción adecuada para este sistema y por qué las alternativas gratuitas encajan mal. Las condiciones de los planes gratuitos cambian con frecuencia: *verifíquense en cada proveedor antes de decidir*.

8.1 El requisito de fondo

TacamaMesa necesita, **encendidos todo el tiempo**: 6 procesos Node, un servidor WebSocket, RabbitMQ, Redis y PostgreSQL con 6 esquemas. Son ~11 procesos que se comunican por red interna y que no pueden "dormirse": la cocina debe recibir los pedidos al instante.

8.2 Supabase

Supabase ofrece **PostgreSQL administrado** más autenticación, almacenamiento, realtime y funciones. Es excelente cuando la aplicación se apoya en esos servicios. Aquí encaja solo en parte:

- **No aloja los 6 microservicios NestJS.** Habría que hospedarlos igualmente en otro lado; Supabase resolvería únicamente la base de datos.
- **No incluye RabbitMQ ni Redis** como los usa el sistema (colas AMQP con exchanges y caché). Habría que contratar servicios aparte.
- **Auth y Realtime propios** duplicarían lo que ya implementan `auth-service` (JWT y roles propios) y Socket.IO. Adoptarlos implicaría **rediseñar**, no solo desplegar.
- Una base remota separada de los servicios añade **latencia** en cada consulta y obliga a gestionar límites de conexiones.
- Los proyectos del plan gratuito inactivos pueden **pausarse**, algo inaceptable en un restaurante.

Conclusión: sería una opción válida como *solo base de datos* en una arquitectura híbrida, pero no ahorra el resto de la infraestructura y suma complejidad.

8.3 Otras plataformas gratuitas o de nivel gratuito

Opción	Por qué encaja mal con este sistema
Render, Railway, Fly.io (planes gratuitos o de prueba)	Los servicios gratuitos suelen **suspenderse por inactividad** y tardar segundos en despertar (arranque en frío), lo que rompe el tiempo real; tienen poca RAM por servicio y límites de horas o créditos. Con 6 servicios + datos, el consumo supera lo gratuito. Varias ya no ofrecen nivel gratuito permanente.
Neon, CockroachDB, PlanetScale…	Resuelven solo la base de datos y con límites de tamaño o de cómputo en gratuito; no sustituyen servicios ni mensajería.
CloudAMQP / Upstash (RabbitMQ / Redis gestionados)	Útiles para pruebas, pero con cuotas bajas de mensajes y conexiones. Usarlos obligaría a partir el sistema en muchos proveedores.
Funciones serverless (Vercel/Netlify Functions, AWS Lambda)	No mantienen conexiones persistentes: **no admiten el WebSocket de cocina** y exigirían reescribir los servicios.

8.4 Qué aporta el VPS

- **Encaja con la arquitectura tal cual está:** el mismo `docker compose` que se usa en desarrollo levanta todo en producción.
- **Siempre encendido**, sin arranques en frío: pedidos y cocina en tiempo real.

- **Un solo lugar, un costo fijo y predecible**, con todos los datos bajo control del equipo (relevante por los datos de alergias).
- **Red interna privada:** bases de datos y mensajería no se exponen a Internet.
- **Portabilidad:** al ser Docker Compose, el sistema puede moverse a cualquier otro VPS con unos pocos comandos.

8.5 Por qué Contabo en particular, y su contrapartida

Entre los VPS (Contabo, Hetzner, DigitalOcean, etc.) cualquiera habría funcionado. Contabo suele destacar por la **cantidad de RAM y disco por precio**, útil aquí porque el sistema corre ~11 contenedores. A cambio, es común que ofrezca menos soporte y un rendimiento de red o disco más variable que otros proveedores. Si el sistema pasara a un uso comercial exigente, convendría revisar esa elección.

> **Contrapartidas del VPS (no es gratis ni "sin mantenimiento").** Es un **punto único de falla**; el equipo debe encargarse de actualizaciones del sistema, respaldos, monitoreo y seguridad. Las secciones 11 y 12 recomiendan cómo mitigarlo.

9. ¿Por qué Vercel y no otras plataformas?

> **Transparencia.** Vercel no era la única opción viable. Se eligió por encajar muy bien con la naturaleza de los frontends y por su sencillez. Los límites de su plan gratuito también deben verificarse en sus condiciones vigentes.

9.1 Qué hay que alojar

Las dos aplicaciones (comensal y personal) son **sitios estáticos**: React compilado con Vite en archivos HTML, JavaScript y CSS. No ejecutan código de servidor. Es el caso ideal para una red de distribución de contenido (CDN).

9.2 Qué aporta Vercel

- **Despliegue continuo desde GitHub:** cada cambio en la rama de producción se compila y publica solo.
- **HTTPS y CDN global incluidos:** carga rápida desde el celular del comensal, sin configurar certificados.
- **Rewrites (proxy) por archivo de configuración:** permite que el navegador llame a `/api/v1/...` en su mismo dominio y Vercel lo reenvíe al VPS. Se evitan problemas de CORS y de contenido mixto.
- **Detección de Vite y de monorepo** con poca configuración; vistas previas por rama.
- **Plan gratuito (Hobby)** suficiente para un proyecto académico.

9.3 Comparación con las alternativas

Plataforma	Valoración para este proyecto
Netlify	**Equivalente.** También ofrece CDN, despliegue desde Git y rewrites. Habría servido igual.
Cloudflare Pages	**Muy buena alternativa**, con CDN potente. Los rewrites hacia un backend externo requieren algo más de configuración.
GitHub Pages	**Limitada.** No admite rewrites hacia el backend: habría que habilitar CORS y llamar al VPS directamente desde el navegador.
Servir los frontends desde el propio VPS (con Caddy)	**Posible y más simple en redes.** Todo en un dominio, sin CORS ni rewrites. Pierde la CDN y concentra todo en un solo servidor.
AWS S3 + CloudFront	**Más trabajo de configuración** para el mismo resultado; no justificado a esta escala.

9.4 Limitaciones a tener en cuenta

- **Plan Hobby = uso personal y no comercial.** Es válido para un proyecto académico, pero la operación comercial del restaurante requeriría un plan de pago o migrar a otra plataforma.
- **Los rewrites no transportan WebSockets:** por eso el socket de cocina va directo al VPS.
- **Dependencia de la cuenta y del repositorio:** hoy los proyectos están en la cuenta de un compañero y conectados a su *fork*. Los cambios solo llegan a Vercel cuando ese fork se sincroniza. Lo ideal es que Vercel se conecte al repositorio original o que el equipo tenga sus propios proyectos.
- **Las variables de entorno** (`VITE_*`) se incorporan al compilar: cambiarlas exige un nuevo despliegue.

10. Validación realizada

Tras el despliegue se hicieron pruebas de extremo a extremo sobre los dominios reales.

Prueba	Resultado
Carta a través de Vercel (104 platos)	**Correcto**
Sesión de mesa por QR a través de Vercel	**Correcto**
Recorrido del comensal: consentimiento, perfil, agregar plato, confirmar pedido	**Correcto**
WebSocket a cocina: evento `nueva_comanda` tras confirmar	**Recibido en ~300 ms**
Portal del personal: KDS muestra "WebSocket conectado" y los pedidos	**Correcto**
Mesas y mozos: mesas ocupadas con su comanda	**Correcto**
Avance de estados desde cocina hasta "Servida"	**Reflejado en el comensal**
Recarga de página con pedido activo	**Conserva el seguimiento (arreglado y verificado)**
Recarga con pedido ya cerrado	**Vuelve a la carta limpia**
Fotos de platos y rutas internas (fallback de SPA) en ambas apps	**Correcto**
Respaldo de la base de datos (`pg_dump`)	**Generado y copiado fuera del servidor**

Problemas encontrados y resueltos durante el despliegue

- **Gateway devolvía texto en vez de la API:** una línea `respond` sin condición en Caddy se ejecutaba antes que el proxy. Se acotó a la ruta raíz.
- **Vercel desplegaba desde otro fork/rama:** estaba conectado a un fork desactualizado y a una rama con otro backend. Se sincronizó y se verificó que producción use `main`.
- **El seguimiento se perdía al recargar:** el identificador del pedido vivía solo en memoria. Ahora se guarda por mesa en `sessionStorage` y se descarta al cerrarse o si no existe.
- **Carpeta duplicada subida por error al repositorio:** se retiró con un commit de corrección.

11. Riesgos, limitaciones y recomendaciones

#	Riesgo	Recomendación
1	**Contraseñas del seed públicas en el repositorio**	Cambiarlas ya desde el módulo Cuentas. Opcional: retirarlas del README público.
2	**Contraseña de root compartida; acceso por contraseña**	`passwd` ahora; pasar a claves SSH y desactivar el acceso por contraseña.
3	**Un solo servidor (punto único de falla)**	Respaldos programados y copia fuera del VPS; plan de restauración documentado; monitoreo básico.
4	**Dependencia del fork y de la cuenta de Vercel de un compañero**	Conectar Vercel al repositorio original o crear proyectos propios; documentar quién administra qué.
5	**Dominio `sslip.io` (depende de un servicio de terceros)**	Comprar un dominio propio y apuntarlo a la IP; actualizar `DOMAIN`, rewrites y `VITE_SOCKET_URL`.
6	**Vercel Hobby es de uso no comercial**	Para operación comercial: plan de pago o alojar los frontends en el VPS con Caddy.
7	**Mantenimiento del sistema operativo y de Docker**	Actualizaciones periódicas de seguridad y reinicios programados; revisar logs.
8	**Cumplimiento legal (Ley N.º 29733)**	Revisar con un abogado la política, los plazos y el registro del banco de datos antes de operar con clientes reales.
9	**Datos de prueba en producción y stock descontado por pruebas**	Revisar indicadores e inventario; reponer existencias y, si se desea, depurar las comandas de prueba.
10	**Valores nutricionales y alérgenos son referenciales**	Validarlos con la cocina o un nutricionista con las recetas reales.

12. Anexo: operación diaria y estructura

12.1 Direcciones

Elemento	Dirección
Comensal	https://sistema-tacama.vercel.app/?mesa=QR-MESA-001 (hasta 006)
Portal del personal	https://sistema-tacama-staff-web.vercel.app
API (backend)	https://13-140-174-57.sslip.io/api/v1/carta
Repositorio	https://github.com/ARamosLoo/SistemaTacama (original); el fork conectado a Vercel es VO71/SistemaTacama

12.2 Comandos de operación (en el VPS, dentro de `~/SistemaTacama`)

```
git pull                                                  # traer cambios
docker compose -f docker-compose.prod.yml up -d --build      # reconstruir y levantar
docker compose -f docker-compose.prod.yml ps                 # estado de contenedores
docker compose -f docker-compose.prod.yml logs --tail 50 comandas-service
docker compose -f docker-compose.prod.yml restart caddy      # reiniciar un servicio
# Respaldo de la base de datos
docker compose -f docker-compose.prod.yml exec postgres pg_dump -U tacama_user tacamamesa > respaldo.sql
```

12.3 Variables de entorno principales (`.env` del VPS)

Variable	Uso
`DOMAIN`	Dominio HTTPS que sirve Caddy.
`POSTGRES_*`, `REDIS_PASSWORD`, `RABBITMQ_*`	Credenciales de los servicios de datos (internos).
`JWT_SECRET`, `JWT_MESA_SECRET`	Firma de tokens del personal y de mesa.
`CORS_ORIGINS`	Dominios de Vercel autorizados.
`VITE_SOCKET_URL` (en Vercel)	Dirección del WebSocket de cocina.

12.4 Estructura del repositorio

```
tacamamesa/
├─ apps/            comensal-pwa · staff-web
├─ services/        auth · catalogo-i18n · maridaje · comandas · inventario · analitica-auditoria
├─ packages/        shared-types · shared-errors · shared-utils
├─ infra/           caddy · docker (Dockerfile.prod) · postgres · rabbitmq · sandbox · data
├─ docs/            arquitectura · privacidad · demo · guías de despliegue (Vercel y VPS)
├─ docker-compose.yml        desarrollo local
└─ docker-compose.prod.yml   producción (VPS)
```

12.5 Documentos relacionados

- `docs/arquitectura_tacamamesa.md` — arquitectura empresarial (TOGAF ADM).
- `docs/PRIVACIDAD.md` — marco legal y pendientes de protección de datos.
- `docs/DESPLIEGUE-VPS.md` y `docs/DESPLIEGUE-VERCEL.md` — guías paso a paso.
- `docs/DEMO.md` — guion de demostración.

© 2026 Proyecto académico desarrollado por estudiantes de la Universidad Tecnológica del Perú (UTP). Sin fines comerciales. La marca, la carta y las imágenes de El Tambo de Tacama pertenecen a sus titulares y se usan solo con fines académicos.